

Análisis Comparativo Profundo entre SOA y Microservicios con Aplicación al Sistema de Control de Laboratorios Informáticos

Freddy Vladimir Farinango Guandinango
Asignatura: Aplicaciones Distribuidas | Junio 2026

Abstract—El presente trabajo realiza un análisis comparativo profundo entre la Arquitectura Orientada a Servicios (SOA) y la Arquitectura de Microservicios, dos paradigmas fundamentales en el diseño de sistemas distribuidos. A través de una revisión sistemática de literatura académica reciente y un análisis crítico propio, se examinan las dimensiones técnicas, operativas y económicas de ambos enfoques, incluyendo escalabilidad, gobernanza, seguridad, mantenimiento y costo de implementación. Los resultados del análisis se aplican al Sistema de Control de Laboratorios Informáticos (SCLI), un proyecto integrador de mediana escala que gestiona reservas, disponibilidad, usuarios, equipos e incidencias en entornos académicos. Se propone y justifica técnicamente una arquitectura de microservicios modulares como solución óptima para el contexto del sistema estudiado.

Palabras clave—SOA, microservicios, arquitecturas distribuidas, servicios web, sistemas académicos, escalabilidad, gobernanza, seguridad.

I. INTRODUCCIÓN

El diseño de sistemas distribuidos ha evolucionado considerablemente en las últimas dos décadas. La Arquitectura Orientada a Servicios (SOA), consolidada a inicios de los años 2000, propuso organizar la funcionalidad de los sistemas empresariales en servicios reutilizables, interoperables y de grano grueso, comunicados a través de protocolos estándar como SOAP y WSDL. Posteriormente, la Arquitectura de Microservicios emergió como respuesta a las limitaciones operativas y organizacionales de SOA, promoviendo la descomposición de aplicaciones en unidades funcionales autónomas, de menor tamaño y desplegadas de forma independiente [1].

Ambos paradigmas comparten el principio de separación de responsabilidades, pero difieren profundamente en su granularidad, modelo de gobernanza, patrones de comunicación y complejidad operativa. Esta divergencia genera debates académicos y prácticos aún vigentes sobre cuál enfoque resulta más adecuado para distintos contextos. Para sistemas de tamaño medio como plataformas académicas institucionales, la elección arquitectónica tiene implicaciones directas en los costos de desarrollo, la facilidad de mantenimiento y la capacidad de evolución del sistema.

El Sistema de Control de Laboratorios Informáticos (SCLI) constituye un caso de estudio representativo: un sistema con múltiples módulos funcionales interrelacionados —reservas, disponibilidad, usuarios, equipos, incidencias y reportes—, que opera en un entorno institucional con recursos limitados pero con demandas de disponibilidad y trazabilidad relativamente altas. El objetivo de este documento es analizar comparativamente SOA y microservicios, evaluar sus características técnicas y limitaciones reales, y proponer una decisión arquitectónica fundamentada para el SCLI.

II. METODOLOGÍA

El análisis se realizó mediante una revisión sistemática de literatura académica indexada en IEEE Xplore, Springer y MDPI, con criterios de inclusión que exigen artículos publicados entre 2022 y 2025, con DOI verificable y revisión por pares. Se seleccionaron seis referencias primarias que aportan evidencia empírica y revisiones sistemáticas relevantes para los ejes de

comparación definidos: rendimiento, escalabilidad, seguridad, gobernanza y migración.

Los ejes de comparación fueron estructurados según las dimensiones técnica (rendimiento, escalabilidad, comunicación), operativa (gobernanza, despliegue, mantenimiento) y de riesgo (seguridad, costo, complejidad). Para la sección de aplicación al SCLI, se analizaron los quince módulos funcionales del sistema y se propuso una descomposición en servicios evaluando ambas alternativas arquitectónicas.

III. RESULTADOS Y DISCUSIÓN

A. Fundamentos de SOA

SOA es un paradigma arquitectónico que estructura una aplicación como un conjunto de servicios de negocio de grano grueso, expuestos mediante interfaces estándar e interoperables. Sus principios fundamentales incluyen la reutilización de servicios, el acoplamiento débil entre componentes, la abstracción de la implementación y la composición de servicios para construir procesos de negocio complejos [2]. En SOA, los servicios encapsulan procesos de negocio completos y se comunican a través de un Enterprise Service Bus (ESB), que centraliza la mediación, transformación y enrutamiento de mensajes.

La gobernanza constituye uno de los pilares de SOA. Un modelo de gobernanza maduro define políticas de ciclo de vida de servicios, contratos de interfaz, registros de servicios y mecanismos de versionado. Esta estructura, aunque robusta para entornos empresariales grandes con múltiples equipos y sistemas legados, impone una carga organizacional significativa. El ESB se convierte frecuentemente en un componente de alta criticidad: su fallo afecta a todos los servicios que dependen de él, constituyendo un punto único de fallo a nivel de integración.

Desde una perspectiva práctica, SOA sigue siendo relevante en entornos donde la integración de sistemas heterogéneos legados es prioritaria. En organizaciones con sistemas ERP, CRM o plataformas gubernamentales, SOA provee los mecanismos de interoperabilidad necesarios sin requerir la reescritura completa de los componentes existentes.

B. Fundamentos de Microservicios

La arquitectura de microservicios descompone una aplicación en servicios pequeños, autónomos y centrados en una única

capacidad de negocio. Cada microservicio posee su propia base de datos, puede ser desplegado y escalado independientemente, y se comunica con otros servicios mediante APIs REST o mecanismos de mensajería asíncrona. Este enfoque se popularizó a partir de su adopción por compañías como Amazon, Netflix y Uber como respuesta a los cuellos de botella de sus arquitecturas monolíticas [1].

La evidencia empírica sobre el rendimiento de microservicios es matizada. Blinowski et al. [1] demostraron que, en máquinas individuales, una arquitectura monolítica supera en rendimiento a su equivalente basado en microservicios. Sin embargo, bajo condiciones de carga distribuida y escalado horizontal, los microservicios exhiben ventajas claras en throughput y tolerancia a fallos.

Söylemez et al. [3] identificaron, a través de una revisión sistemática de 58 artículos, que los principales desafíos de microservicios incluyen la gestión de la consistencia de datos distribuidos, la complejidad del despliegue, la latencia de comunicación inter-servicio y la dificultad de pruebas de integración. Una limitación poco discutida es el denominado *distributed systems tax*: el costo cognitivo y técnico de razonar sobre fallos parciales, particiones de red y consistencia eventual en un sistema compuesto por decenas de servicios.

C. Comparación Técnica

Granularidad y acoplamiento. SOA opera con servicios de grano grueso que encapsulan procesos de negocio completos, lo que facilita la comprensión del sistema pero incrementa la superficie de impacto ante cambios. Los microservicios, al ser de grano fino, permiten modificaciones aisladas sin afectar otras partes del sistema.

Comunicación. SOA utiliza predominantemente el ESB como intermediario centralizado, empleando SOAP/WSDL para contratos de servicio. Los microservicios favorecen la comunicación directa mediante REST o gRPC para interacciones síncronas, y sistemas de mensajería como Kafka o RabbitMQ para comunicación asíncrona basada en eventos.

Escalabilidad. En SOA, la escalabilidad se aplica a nivel de todo el servicio o del ESB, implicando escalar componentes enteros aunque solo una función específica sea el cuello de botella. Los microservicios permiten escalar selectivamente los componentes con mayor demanda, optimizando el uso de recursos computacionales [1].

Gobernanza y mantenimiento. SOA impone un modelo de gobernanza centralizada con registros de servicios, contratos formalizados y comités de arquitectura. Los microservicios favorecen la gobernanza descentralizada, siguiendo el principio de *smart endpoints, dumb pipes*. Capuano y Muccini [2] observaron que la migración hacia microservicios está frecuentemente motivada por la necesidad de mejorar la mantenibilidad y la desplegabilidad en equipos que adoptan prácticas DevOps y CI/CD.

Seguridad. En SOA, la seguridad se gestiona centralizadamente a través del ESB o de un Identity Provider. En microservicios, la superficie de ataque se incrementa debido a la proliferación de puntos de comunicación inter-servicio. Haindl

et al. [4] identificaron que los vectores más críticos incluyen la suplantación de identidad entre servicios, la interceptación de comunicaciones y el escalado de privilegios mediante tokens comprometidos.

Costos de implementación. La infraestructura necesaria para microservicios requiere inversión significativa en tiempo y conocimiento. Para proyectos académicos o de pequeña escala, este overhead puede superar los beneficios obtenidos.

Table 1: COMPARACIÓN TÉCNICA: SOA VS. MICROSERVICIOS

Dimensión	SOA	Microservicios
Granularidad	Grano grueso (procesos)	Grano fino (capacidades atómicas)
Comunicación	ESB / SOAP / WSDL	REST / gRPC / Mensajería asíncrona
Escalabilidad	A nivel de servicio o ESB	Selectiva por componente
Gobernanza	Centralizada, formal	Descentralizada, autónoma
Seguridad	Centralizada en ESB/IdP	Distribuida, mayor superficie
Despliegue	Tradicional, infrecuente	Contenedores, CI/CD frecuente
Costo inicial	Alto (licencias)	Alto (orquestración, observab.)
Costo operativo	Alto (mantenimiento ESB)	Medio-Alto (complejidad dist.)
Madurez	Alta	Alta

D. Análisis Crítico Propio

Aunque la literatura destaca los microservicios como el estándar arquitectónico moderno, se considera que esta narrativa requiere matización cuando se aplica a contextos específicos. El entusiasmo por microservicios en la industria de tecnología proviene principalmente de organizaciones con equipos de decenas o cientos de ingenieros, infraestructuras cloud maduras y sistemas con millones de usuarios concurrentes.

Una limitación poco discutida de los microservicios es lo que podría denominarse la *trampa de la granularidad prematura*: la tendencia a descomponer funcionalidades que no han demostrado necesitar despliegue independiente, generando overhead de comunicación y complejidad de gestión sin beneficios operativos reales.

Desde una perspectiva práctica, la elección entre SOA y microservicios no debería ser un juicio de valor arquitectónico en abstracto, sino una decisión técnica contextualizada que pondere el tamaño y experiencia del equipo, la carga concurrente esperada, el presupuesto de infraestructura y la velocidad de cambio requerida.

IV. APLICACIÓN AL PROYECTO INTEGRADOR SCLI

A. Contexto y Caracterización del Sistema

El SCLI es una plataforma de gestión institucional que integra quince módulos funcionales: asignación de laboratorios, gestión de disponibilidad, control de ocupación, reservas, cancelación de reservas, control de listas, gestión de roles de usuario, registro de equipos informáticos, gestión de incidencias, reportes administrativos, administración de usuarios, control de acceso, historial de actividades, monitoreo del estado de equipos y generación de reportes operativos.

Desde una perspectiva de diseño arquitectónico, estos módulos exhiben características heterogéneas en cuanto a frecuencia de cambio, carga de uso y aislamiento funcional. El control de acceso y la gestión de usuarios son altamente transversales y requieren disponibilidad permanente. La generación de reportes y el historial de actividades son de uso esporádico y tolerantes a mayor latencia.

B. Implementación bajo SOA

Bajo una arquitectura SOA, el SCLI se estructuraría en cuatro servicios de grano grueso. El *Servicio de Gestión de Espacios* encapsularía la asignación de laboratorios, gestión de disponibilidad, control de ocupación y gestión de reservas, exponiendo operaciones CRUD con contratos WSDL. El *Servicio de Gestión de Usuarios y Acceso* integraría la administración de usuarios, gestión de roles y control de acceso, siendo consumido por todos los demás servicios a través del ESB mediante WS-Security y tokens SAML.

El *Servicio de Inventario y Equipos* abarcaría el registro de equipos informáticos, monitoreo del estado de equipos y gestión de incidencias. El *Servicio de Trazabilidad y Reportes* consolidaría el historial de actividades, control de listas y reportes operativos. En el contexto académico del SCLI, esta arquitectura implicaría la instalación de un ESB como WSO2 API Manager, lo que demanda conocimiento especializado y recursos computacionales prohibitivos para un equipo reducido.

C. Implementación bajo Microservicios

Bajo una arquitectura de microservicios, el SCLI se descompondría en diez unidades autónomas: *reservation-service*, *availability-service*, *occupancy-service*, *user-service*, *auth-service*, *equipment-service*, *incident-service*, *reporting-service*, *audit-service* y *notification-service*. La comunicación emplearía un API Gateway como punto de entrada único con REST/JSON para operaciones síncronas y RabbitMQ para comunicaciones asíncronas. El *auth-service* implementa JWT de corta expiración con refresh tokens, mitigando los riesgos de seguridad identificados por Haindl et al. [4] y Berardi et al. [5].

D. Propuesta Arquitectónica Justificada

Se considera que la arquitectura más adecuada para el SCLI es una **arquitectura de microservicios selectiva** —microservicios modulares—, que aplica los principios de microservicios únicamente donde el aislamiento funcional aporta valor real, manteniendo servicios de granularidad media en los dominios más estables.

La propuesta concreta estructura el SCLI en cinco servicios

desplegables de forma independiente:

1. **Servicio de Reservas y Disponibilidad:** núcleo funcional de mayor frecuencia de cambio, agrupa reservas, disponibilidad y ocupación.
2. **Servicio de Identidad y Control de Acceso:** transversal al sistema con alta disponibilidad, implementado con JWT y RBAC.
3. **Servicio de Inventario:** dominio estable que agrupa equipos e incidencias.
4. **Servicio de Reportes y Auditoría:** dominio consultivo tolerante a latencia, implementado con CQRS sobre datos replicados.
5. **API Gateway:** provee punto de entrada único y desacopla los clientes de los servicios internos.

Esta propuesta reduce la complejidad operativa respecto a diez microservicios, manteniendo los beneficios de aislamiento en los dominios más críticos. Para el contexto académico del SCLI con equipos de tres a cinco desarrolladores, la propuesta modular ofrece el mejor balance entre modularidad técnica, mantenibilidad operativa y viabilidad de implementación.

V. CONCLUSIONES

Primera. SOA y microservicios no constituyen paradigmas sucesivos donde uno reemplaza al otro, sino enfoques complementarios con contextos de aplicación diferenciados. SOA sigue siendo relevante para la integración de sistemas heterogéneos con componentes legados, mientras que los microservicios aportan mayor agilidad y aislamiento de fallos en sistemas nuevos con equipos autónomos y prácticas DevOps maduras.

Segunda. La evidencia empírica matiza la superioridad de microservicios en rendimiento. Los beneficios de escalabilidad y tolerancia a fallos se materializan bajo condiciones de carga distribuida real; bajo carga moderada, los servicios de grano grueso pueden exhibir mejor rendimiento con menor complejidad [1].

Tercera. La seguridad en arquitecturas de microservicios requiere atención específica en el perímetro de comunicación inter-servicio. La implementación de mTLS, JWT con scopes granulares y un API Gateway centralizado es condición necesaria, no opcional [4,5].

Cuarta. Para el SCLI, la arquitectura de microservicios modulares propuesta —cinco servicios agrupados por dominio funcional— ofrece el balance óptimo entre aislamiento y viabilidad operativa en un contexto académico con recursos limitados. Esta arquitectura puede evolucionar progresivamente hacia una descomposición más granular conforme el sistema y el equipo maduren.

Quinta. La decisión arquitectónica entre SOA y microservicios no debe fundamentarse exclusivamente en criterios técnicos o tendencias de la industria, sino en un análisis contextualizado que considere el tamaño del equipo, la carga esperada, el presupuesto de infraestructura y la velocidad de cambio requerida.

REFERENCES

- [1] G. Blinowski, A. Ojdowska, y A. Przybyłek, “Monolithic vs. Microservice Architecture: A Performance and Scalability Evaluation,” *IEEE Access*, vol. 10, pp. 20357–20374, 2022, doi: 10.1109/ACCESS.2022.3152803.
- [2] R. Capuano y H. Muccini, “A Systematic Literature Review on Migration to Microservices: A Quality Attributes Perspective,” en *2022 IEEE 19th Int. Conf. Software Architecture Companion (ICSA-C)*, Honolulu, HI, USA, 2022, pp. 120–123, doi: 10.1109/ICSA-C54293.2022.00030.
- [3] M. Söylemez, B. Tekinerdogan, y A. Kolukısa Tarhan, “Challenges and Solution Directions of Microservice Architectures: A Systematic Literature Review,” *Applied Sciences*, vol. 12, no. 11, p. 5507, 2022, doi: 10.3390/app12115507.
- [4] P. Haindl, P. Kochberger, y M. Sveggen, “A Systematic Literature Review of Inter-Service Security Threats and Mitigation Strategies in Microservice Architectures,” *IEEE Access*, vol. 12, pp. 90252–90286, 2024, doi: 10.1109/ACCESS.2024.3406500.
- [5] D. Berardi, S. Giallorenzo, J. Mauro, A. Melis, F. Montesi, y M. Prandini, “Microservice Security: A Systematic Literature Review,” *PeerJ Computer Science*, vol. 8, p. e779, 2022, doi: 10.7717/peerj-cs.779.
- [6] B. Prasetyo, R. Hatta, y A. Firmansyah, “Evaluating Middleware Performance in the Transition from Monolithic to Microservices Architecture for Banking Applications,” *Electronics (MDPI)*, vol. 15, no. 1, p. 221, 2026, doi: 10.3390/electronics15010221.